

毕业设计进度汇报稿

题目：基于 OpenHands SDK 的 Multi-Agent FPGA 自动化设计系统 汇报人：贾阔源 | 指导老师：刘亚男 日期：2026 年 4 月

Slide 1 — 封面页

【展示】 题目 + 姓名 + 指导老师

【讲稿】

各位老师，我是贾阔源，我的毕业设计题目是"基于 OpenHands SDK 的 Multi-Agent FPGA 自动化设计系统"，指导老师是刘亚男老师。接下来我将从研究背景、关键技术框架、项目具体实施和总结展望四个部分，汇报当前的工作进展。

Slide 2 — 目录页

【展示】 四个章节目录：①研究背景及意义 ②关键技术框架 ③项目具体实施 ④总结与展望

【讲稿】

本次汇报分为四个部分。首先介绍研究背景及意义，说明为什么要做这个课题；然后介绍两个关键技术框架——OpenHands SDK 和 DeepSeek 大语言模型；第三部分是项目的具体实施方案，也是本次汇报的重点；最后进行总结与展望。

Slide 3 — 大语言模型发展的关键节点（时间线）

【展示】 LLM 发展时间线：20世纪 → 2003 → 2014 → 2015 → 2017 → 2018 → 2024 → 2025

【讲稿】

首先回顾大语言模型的发展历程。从20世纪早期基于统计的n-gram模型，到2003年Bengio团队提出神经网络语言模型，再到2014年Seq2Seq框架和2015年注意力机制（Attention）的提出，这些工作奠定了序列建模的基础。

真正的革命性突破发生在2017年——Google提出了Transformer架构，它摒弃了RNN的顺序计算限制，通过自注意力机制实现了全并行训练，成为后续所有大模型的基石。2018年，基于

Transformer的GPT和BERT相继问世，开启了预训练大模型时代。到2024年，国产大模型DeepSeek横空出世，以其独特的MoE架构和极具竞争力的性价比迅速成为业界关注的焦点。2025年被业界称为"智能体工程元年"，大量Agent框架涌现，这也是我们这个课题立项的大背景。

Slide 4 — 大语言模型发展的关键节点（补充图示）

【展示】LLM发展关键节点的补充图片/架构图

【讲稿】

这页展示了几个关键模型的架构演进。可以看到，从早期的RNN到Transformer，再到如今GPT-4、Claude、DeepSeek等千亿参数级别的模型，大语言模型在代码生成能力上取得了突破性进展。这些模型不仅能够生成逻辑正确的高级语言代码，也逐步展现出生成硬件描述语言（Verilog/VHDL）的能力，这为我们将LLM应用于FPGA设计自动化提供了技术基础。

Slide 5 — 大语言模型在EDA领域的应用

【展示】EDA领域LLM应用的三个维度（Understanding、Coding、Refinement）+ 两篇代表论文

【讲稿】

这页展示了LLM在EDA（电子设计自动化）领域的应用现状。当前的研究工作主要围绕三个维度展开：

1. Understanding——用LLM



【讲稿】

下面进入第二部分——关键技术框架。我将介绍本项目依赖的两个核心技术组件：OpenHands SDK 多智能体框架和 DeepSeek 大语言模型。

Slide 7 — OpenHands（概览）

【展示】 OpenHands 平台全景架构图

【讲稿】

OpenHands 是一个开源的AI驱动软件工程平台，也是本项目的运行底座。它提供了 **Agent-Conversation-Tool** 三层核心抽象：

- **Agent层**：支持定义不同角色的智能体，每个Agent有独立的系统提示、工具集和行为策略；
- **Conversation层**：通过事件驱动机制（Action/Observation）管理Agent与环境的交互；
- **Tool层**：支持自定义工具注册，Agent通过工具调用完成具体操作。

我选择OpenHands SDK而非AutoGen等其他框架的核心原因是：**OpenHands原生支持自定义工具注册和MCP（Model Context Protocol）集成**，这使我能够将Verilator仿真器作为Agent可调用的工具接入框架，实现"Agent写代码→框架自动编译仿真→解析结果→反馈Agent"的闭环。

Slide 8 — OpenHands（SDK细节）

【展示】 OpenHands SDK的Agent层级结构和事件系统

【讲稿】

这页展示了OpenHands SDK更具体的结构。在本项目中，我基于SDK构建了**5种Agent角色**：

1. **Workflow Orchestrator**（工作流编排器）——驾驭整个设计流程的全局控制者，使用 thinking 理模式；
2. **Finalizer Orchestrator**（终态编排器）——负责集成回归后的最终总结；
3. **Module Worker**（模块工作者）——负责单个模块的RTL生成和修复，使用fast执行模式；
4. **Repair Worker**（修复工作者）——处理验证失败后的结构化修复；
5. **L2 Campaign Agent**——执行随机和边界条件的鲁棒性测试。

这种分层委派的设计，使得**编排器专注于全局决策，工作器专注于局部执行**，每个角色的权限和行为边界都由框架严格控制。

Slide 9 — DeepSeek (MoE架构)

【展示】 DeepSeek MoE架构图 + Dense MLP对比 + Transformer Block结构

【讲稿】

本项目选择 **DeepSeek Chat** 作为LLM后端。这页展示了DeepSeek的核心架构特点。

DeepSeek采用了**MoE (Mixture of Experts, 混合专家)** 架构。与传统Dense模型不同，MoE通过每次 理仅激活少量专家网络，实现了稀疏计算——在保持模型总容量的同时大幅降低单次理的计算量。

具体来说，DeepSeek的61层Decode层中，前3层使用Dense MLP（选项A），后58层使用MoE结构（选项B）。每个MoE层包含共享专家（Shared Experts）和路由专家（Routed Experts），通过TopK Router动态选择激活哪些专家。

这种架构使DeepSeek在硬件代码生成任务中表现出**优异的性价比**：理能力接近顶级闭源模型，但API调用成本显著更低，非常适合需要大量LLM调用的自动化流水线场景。

Slide 10 — DeepSeek Prefill 阶段数据流

【展示】 Prefill阶段的完整数据流图（含张量维度标注）

【讲稿】

这两页详细展示了DeepSeek 理过程中的数据流。这是**Prefill（预填充）阶段**——处理整个输入序列。

数据流从输入token开始，经过Embedding后进入61层Decode Layer。在Attention模块中，DeepSeek使用了**MLA (Multi-head Latent Attention)** 机制——这是DeepSeek的一大创新：通过将KV投影到低维空间（512维的kv_nope和64维的k_rope），大幅减少了KV Cache的显存占用。

图中可以看到完整的张量维度变化：输入(b, s, 7168)经过Q_down-proj压缩到(b, s, 1536)，再通过Q_up-proj扩展为128个注意力头，每个头192维。MoE层中，TopK Router选择活跃专家后，gate和up分支经过SiLU激活再与down_Linear组合，实现稀疏前馈计算。

在Prefill阶段，所有位置的token并行处理，序列维度s贯穿整个计算图。

Slide 11 — DeepSeek Decode 阶段数据流

【展示】 Decode阶段的完整数据流图（含KV Cache标注）

【讲稿】

这是**Decode（解码）**阶段——自回归地逐个生成token。与Prefill阶段的关键区别有两点：

1. **序列维度从s变为1**：每次只处理一个新token，因此张量从(b, s, 7168)变为(b, 1, 7168)；
2. **引入KV Cache**：图中的"K_cat+K_Cache"和"V+V_Cache"节点表示将当前步的K/V与历史缓存拼接，注意力计算的KV序列长度为s+1。

这种Prefill-Decode分离的架构对于我们的Agent系统来说非常重要：**编排器的系统提示（较长的合约和技能文档）**主要消耗**Prefill**算力，而**Agent的交互式推理（逐步生成代码和决策）**消耗**Decode**算力。DeepSeek的MLA机制通过压缩KV Cache，使得长上下文的多轮对话（Agent修复循环中的典型场景）在算力和显存上都更加高效。

Slide 12 — 目录页（过渡）

【展示】 目录高亮"03 项目具体实施"

【讲稿】

下面进入第三部分——项目具体实施。这是本次汇报的核心内容，我将详细介绍系统架构、工作流程、任务规划与知识注入、批次调度、模块实现、MCP工具仿真和执行器设计。

Slide 13 — 项目架构

【展示】 系统六层架构全景图

【讲稿】

这是项目的整体架构图。整个系统采用**六层分层架构**，从底层到顶层依次是：

1. **Contract Layer（合约层）**：基于Pydantic的数据模型和校验，包括SpecIR（设计规格）、PlanDAG（执行计划）、ModuleContract（模块合约）等。所有合约使用 `extra='forbid'`，拒绝任何未定义字段，这是保证数据正确性的第一道防线。
2. **Synthesis Layer（综合层）**：从YAML蓝图文件或硬编码的AES蓝图生成所有核心制品——SpecIR、PlanDAG和IntegrationManifest。BlueprintLoader负责解析蓝图，

ContractCompiler生成每个节点的合约。

3. **Execution Layer（执行层）**：四个验证执行器——L0编译门控、L1仿真验证、L2鲁棒性测试、Integration集成回归。底层通过VerilatorMCPAdapter与Verilator仿真器交互。
4. **Runtime Layer（运行时层）**：SDK集成和会话编排，包括RuntimeBootstrap初始化、SdkAgentFactory创建各角色Agent、ConversationRunner驱动对话、ExecutionSession管理全流程。
5. **Generation/Workspace Layer（工作区层）**：Memory预填充、结构化修复循环、工作区晋升（promote）生命周期管理。
6. **Agent/Prompt Layer（Agent/提示层）**：角色规格定义、可组合的提示合约（BaseContract + PhaseMixin）、领域技能注入。

这种分层设计的关键优势是**关注点分离**：每一层只负责一个维度的职责，层间通过明确的数据契约（Pydantic模型）通信。

Slide 14 — 项目工作流程

【展示】端到端工作流程图 + 编排状态机

【讲稿】

这页展示了系统的端到端工作流程。核心是一个**10状态**的编排状态机：

```
SPEC_INTAKE → ARCHITECTING → PLANNING → MODULE_DESIGN → MODULE_L0 → MODULE_L1  
→ MODULE_L2_OPTIONAL → INTEGRATION_READY → INTEGRATION_REGRESSION → DONE
```

这里要特别强调**"框架控制"**与**"提示词控制"**的根本区别。在AutoGen等传统方案中，执行进度由LLM的输出决定——LLM说"验证通过了"系统就继续 进，这是**Prompt-controlled**。而在本系统中，**状态机拥有控制权**：每一步状态转移必须满足框架定义的前置条件（如所有Checkpoint通过），LLM无法绕过验证门控"提前宣布成功"。

配合**AgentExecutionPolicy策略层**，每个状态映 到确定的LLM配置和子Agent委派策略：

- 规格摄入、架构设计、集成阶段使用**thinking profile**（深度 理），且**禁止委派子Agent**；
- 模块设计和验证阶段使用**fast profile**（ 速执行），且允许委派模块工作者。

这确保了全局决策使用更强的 理能力，局部执行使用更 的响应速度。

Slide 15 — 任务规划和知识注入

【展示】 任务规划DAG图 + 知识注入流程图

【讲稿】

任务规划的核心是将设计分解为**有向无环图（DAG）**。以AES-128为例，4个模块形成如下依赖关系：



`aes_sbox` 是最底层的叶子节点，被其他三个模块依赖；`aes128_encrypt_core` 是顶层汇聚节点，依赖所有子模块。

知识注入是Harnessing Engine的核心机制之一。具体实现是**Memory预填充**——`MemoryStore` 从Skill文件中检索验证通过的参考RTL和Testbench代码，通过 `populate_workspace()` 写入Agent的工作区。这将Agent的任务从"从零生成正确的RTL代码"降级为"基于验证通过的参考代码进行适配"，显著降低了生成难度。

实验结果证明了这一策略的有效性：全部4个模块均在首次生成后直接通过验证，修复轮次为0，首次通过率100%。

Slide 16 — 批次调度

【展示】 DAG拓扑批调度示意图 + 批次执行甘特图

【讲稿】

DAG拓扑批调度是本系统区别于线性pipeline的关键能力。`DAGBatchPlanner` 使用Kahn算法的变体，将DAG自动分解为多个拓扑层次（batch）：

- **Batch 0 (Layer 0)** : `aes_sbox` —— 无依赖的叶子节点，最先执行；
- **Batch 1 (Layer 1)** : `aes_key_schedule_128` 和 `aes_round_transform` —— 依赖 `aes_sbox`，可并行执行；
- **Batch 2 (Layer 2)** : `aes128_encrypt_core` —— 依赖前面所有模块，最后执行。

每个batch内的节点**独立并行**执行，batch之间**串行门控**——前一个batch的所有模块必须通过验证并晋升（promote）后，下一个batch才会启动。

这里还有一个重要的设计：**文件系统门控（Batch Gate）**。每个batch的"通过"条件不是LLM说通过就通过，而是框架检查promoted目录下是否存在对应模块的验证通过文件。这是框架控制的具体体现——**由文件系统事实而非LLM输出决定批次推进**。

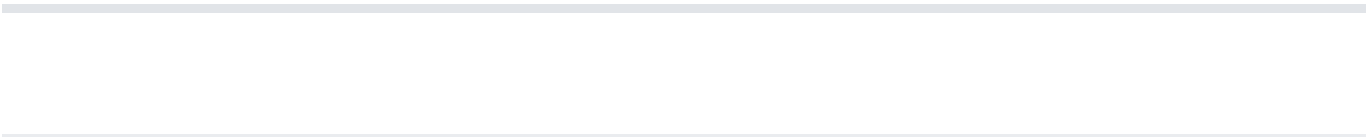
Slide 17 — 子功能模块实现

【展示】 各模块的功能描述和端口定义

【展示】

这里展示了Slide-12中验证案例中4个子模块的具体实现。

- n"-n现



核心组件是 **VerilatorMCPAdapter**——一个异步MCP适配器，负责将框架的验证请求转换为Verilator的编译和仿真操作。它的工作流程是：

1. Agent调用 `run_executor` 自定义工具（这是**唯一合法入口**，Agent不能直接调用Verilator）；
2. `run_executor` 根据任务类型分发到对应的执行器（L0/L1/L2/Integration）；
3. 执行器通过VerilatorMCPAdapter发起MCP调用；
4. Adapter进行**预处理**——过滤非Verilog文件、自动生成 `-I` 包含路径标志、添加C++17编译标志；
5. 通过stdio管道与Node.js MCP Server通信，调用 `verilator_compile` 和 `verilator_simulate`；
6. 仿真完成后，从 `simulation.log` 中解析Checkpoint行，生成结构化的验证结果反馈给Agent。

MCP层还实施工具白名单过滤：只允许 `verilator_compile` 和 `verilator_simulate` 两个工具，屏蔽了 `testbenchgenerator` 和 `naturallanguage` 等危险工具，防止Agent绕过框架的结构化流程。

Slide 19 — Executor（执行器）

【展示】 四层执行器架构图 + `run_executor`工具流程

【讲稿】

这页展示了**四层分级验证执行器**的设计。

- **L0 Executor（编译门控）**：调用Verilator对RTL进行编译检查，捕获语法错误、未声明信号和类型不匹配。这是**秒级的快失败机制**，在仿真之前就拦截明显的代码错误。
- **L1 Executor（仿真+Checkpoint）**：编译成功后执行仿真，解析simulation.log中的 `CHECKPOINT | <name> | PASS | <detail>` 行。只有所有必需Checkpoint都通过，模块才被标记为VALIDATED。
- **L2 Campaign Executor（鲁棒性测试）**：使用随机向量和边界条件测试，验证模块在非标准输入下的行为。通过plusargs传递profile、seed、cases等参数，失败时生成 `counterexample.json` 和 `fragility_summary.json`。
- **Integration Regression Executor（集成回归）**：在所有模块晋升后运行，执行三种Campaign：
 - **baseline**——NIST标准KAT向量，验证基本正确性；

- **back_to_back**——连续多帧加密，验证流水线 and 状态管理；
- **mid_reset**——加密过程中异步复位，验证异常恢复能力。

`run_executor` 是Agent的**唯一执行入口**——它加载任务合约、断执行器类型、分发执行、构建结构化的Observation反馈（包含 `next_read_hints` 指导Agent下一步操作）。这个设计确保了所有验证操作都经过框架的统一管控。

Slide 20 — 目录页（过渡）

【展示】 目录高亮"04 总结与展望"

注：此页目录文本有笔误"总结与总结"，应修正为"总结与展望"。

【讲稿】

最后进入总结与展望部分。我将回顾实验验证结果，总结三个核心创新点，并讨论当前的局限和未来方向。

Slide 21 — 实验案例1：流水灯（简单案例）

【展示】 LED Chaser流水灯设计案例

【讲稿】

为了验证框架的通用性，我们首先用一个**简单案例——LED流水灯**进行了端到端测试。

LED流水灯是一个单模块设计，DAG只有一个节点，没有模块间依赖。这个案例主要验证了：

1. **蓝图驱动的通用性**：通过编写 `led_chaser/blueprint.yaml`，框架无需修改核心代码即可适配新设计；
2. **状态机的基本流转**：从SPEC_INTAKE到DONE的完整生命周期；
3. **L0+L1验证链路**：编译门控和仿真Checkpoint的端到端工作。

流水灯作为"最小可验证设计"，证明了框架的**核心流程不依赖于AES-128的特定实现**——这是架构层通用性的第一步验证。

Slide 22 — 实验案例2：AES加密算法（复杂案例）

【展示】 AES-128全系统验证案例

【讲稿】

AES-128加密全系统是本课题的**核心验证案例**——一个真正的多模块协同设计任务。

实验结果如下：

- **单模块验证**：全部4个模块（aes_sbox、aes_key_schedule_128、aes_round_transform、aes128_encrypt_core）均在**首次生成后直接通过L0编译与L1 Checkpoint验证**，修复轮次为0，**首次通过率100%**；
- **集成回归测试**：3个Campaign（baseline、back_to_back、mid_reset）**全部PASS**，顶层模块6/6 Checkpoint通过，验证了AES-128全系统在标准向量、连续加密和异步复位场景下的正确性；
- **框架控制有效性**：16个执行批次中有11个修复批次因 `condition_false`（首次已通过）被自动跳过，**所有推进决策均由状态机和Checkpoint判定驱动，LLM无法绕过验证门控**。

这些结果证明了本文三个创新点的有效性：**框架控制的编排状态机**确保了流程可靠性；**四层分级验证**确保了结果正确性；**Harnessing Engine**六层约束引擎确保了LLM行为可控性。

总结陈词

【讲稿】

总结一下，本课题针对现有LLM+EDA方案的五个不足——Prompt控制不可靠、线性pipeline无法处理依赖、缺乏结构化修复、验证层级单一、LLM约束工程不足——提出了三个核心创新点：

1. **框架控制的编排状态机**——将控制权从LLM转移到状态机，配合AgentExecutionPolicy实现精确的行为边界控制；
2. **L0→L1→L2→Integration四层分级验证**——从编译到仿真到鲁棒性到集成回归，层层门控，Checkpoint协议实现机器自动判定；
3. **Harnessing Engine六层LLM约束引擎**——Skill/Prompt/Memory/Hooks/MCP/SlashCommand多维度深度防御，覆盖5种典型LLM失败模式。

AES-128全系统的成功验证证明了这一通用框架的有效性。未来的方向包括：扩展到更多硬件设计（AES-256、SHA-256）、引入L3综合阶段、支持多LLM后端、以及Agent自主学习能力。

以上就是我的汇报，谢谢各位老师！

附录：三张补充架构图说明

以下三张Mermaid架构图已编译为PNG，可按需插入PPT中：

图编号	文件	建议插入位置	说明
15	15_mcp_verilator_adapter.png	Slide 18（MCP Tool仿真）	MCP VerilatorMCPAdapter完整数据流
16	16_harnessing_engine.png	新增Slide或Slide 15补充	Harnessing Engine六层约束架构
17	17_core_innovations.png	总结部分（Slide 22之后）	三大核心创新点及循环关系